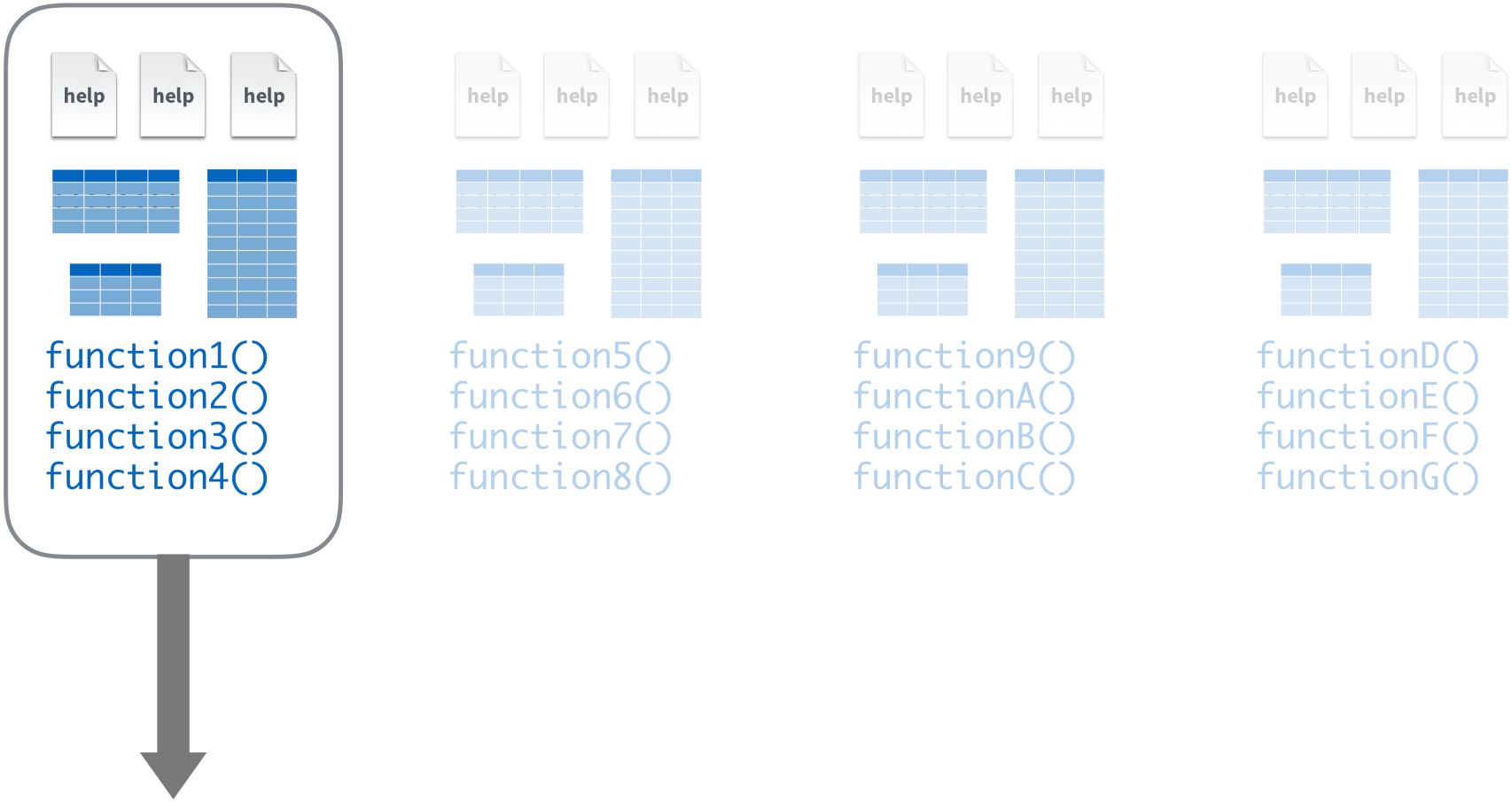
Data manipulation basics

Overview

1. **The `tidyverse`**
   - R Packages
   - Importing data
2. **The `dplyr` package**
   - `filter()`
   - `mutate()`
   - `ifelse()`
   - pipes `|>`
   - `summarize()`
   - `group_by()`
3. **Merge and re-shape**
   - Summary and key takeaways

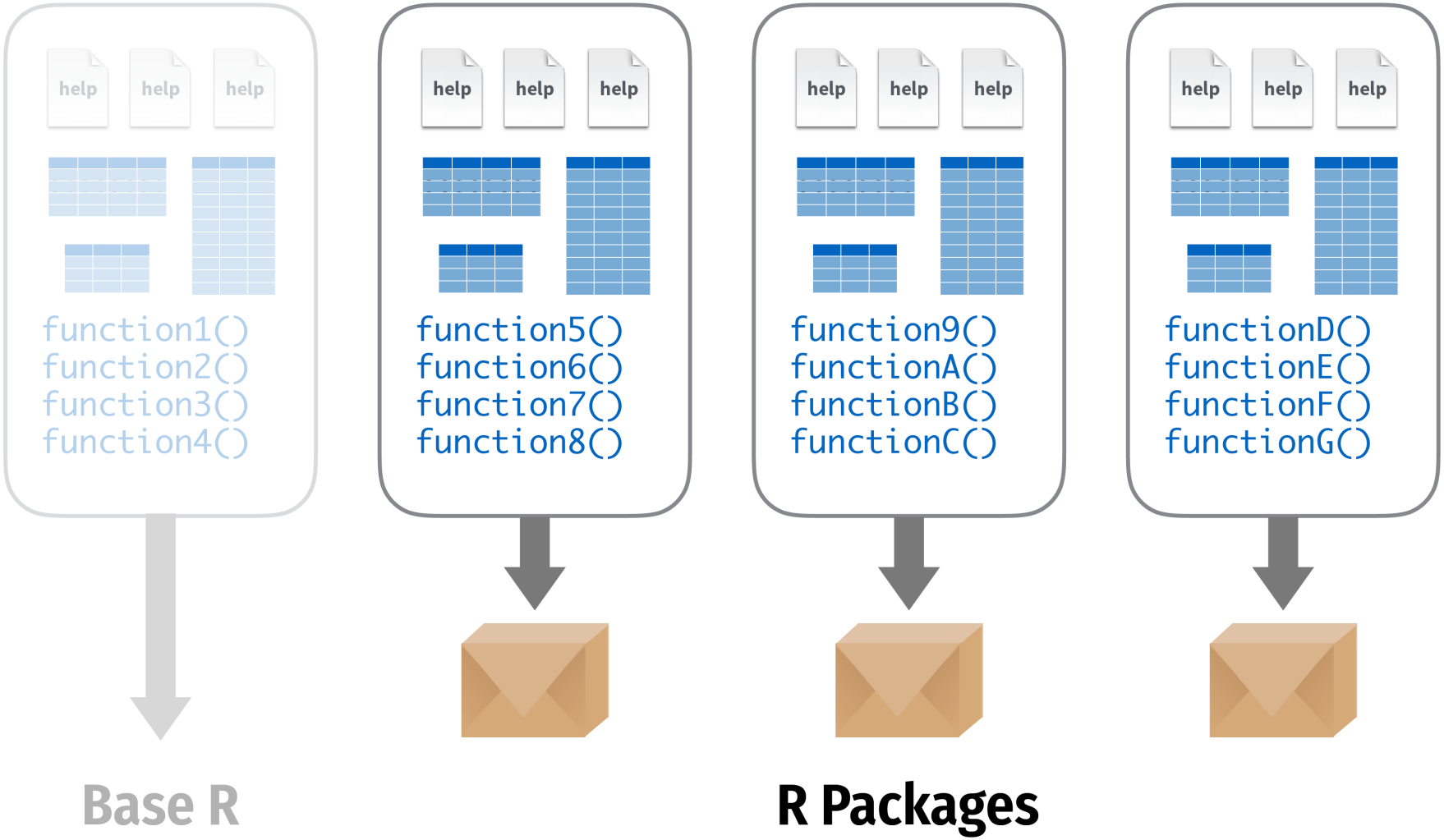
The `tidyverse`

Packages



Base R

Packages


help
help
help
function1()
function2()
function3()
function4()
help
help
help
function5()
function6()
function7()
function8()
help
help
help
function9()
functionA()
functionB()
functionC()
help
help
help
functionD()
functionE()
functionF()
functionG()
Base R
R Packages

Packages

- So far we only used functions that are directly available in R
  - But there are tons of user-created functions out there that can make your life so much easier
  - These functions are shared in what we call packages
- Packages are bundles of functions that R users put at the disposal of other R users
  - Packages are centralized on the Comprehensive R Archive Network (CRAN)
  - To download and install a CRAN package you can simply type `install.packages()

Using packages

```r
install.packages("name")
```

- files to your computer
- Do this once per computer

```r
library("name")
```

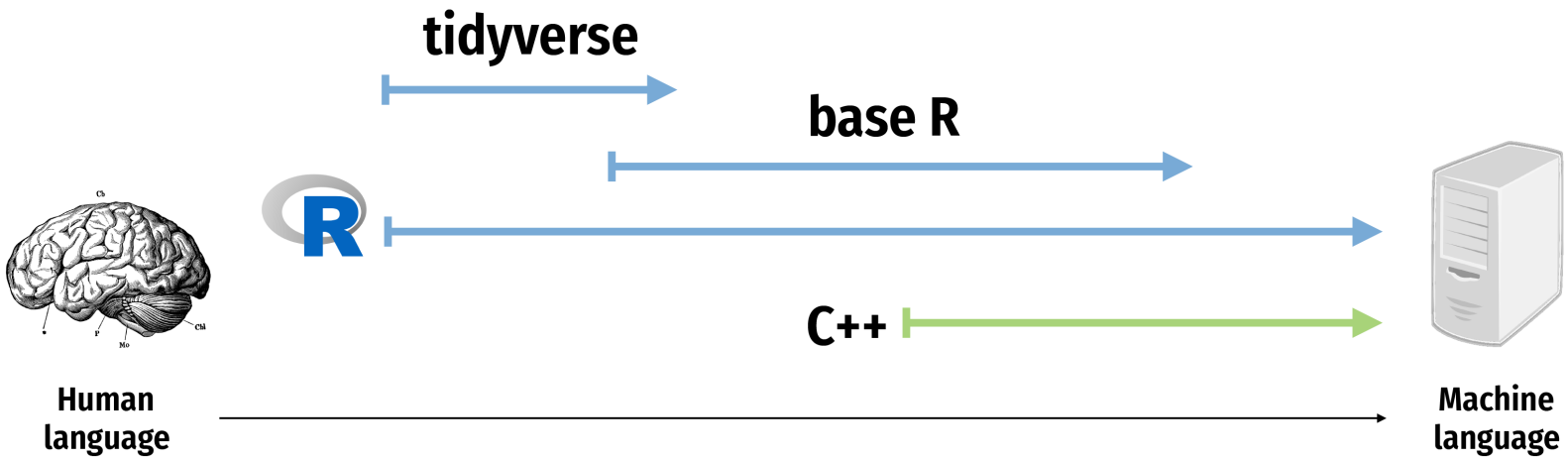
- Loads the package
- Do this once per R session

The `tidyverse`

"The tidyverse is an opinionated collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures."

... the tidyverse makes data science faster, easier and more fun...

The `tidyverse`



The `tidyverse`

```
library(tidyverse)
```

The tidyverse package is a shortcut for installing and loading all the key tidyverse packages

The `tidyverse`

```r
install.packages("tidyverse")
```

```r
library(tidyverse)
```

Installs all of these:

```r
install.packages("ggplot2")
install.packages("dplyr")
install.packages("tidyr")
install.packages("readr")
install.packages("purrr")
install.packages("tibble")
install.packages("stringr")
install.packages("forcats")
install.packages("lubridate")
install.packages("hms")
install.packages("DBI")
install.packages("haven")
install.packages("httr")
install.packages("jsonlite")
install.packages("readxl")
install.packages("rvest")
install.packages("xml2")
install.packages("modelr")
install.packages("broom")
```

Loads all of these:

```r
library(ggplot2)
library(dplyr)
library(tidyr)
library(readr)
library(purrr)
library(tibble)
library(stringr)
library(forcats)
library(lubridate)
```

Importing data

Work with plain text data

```
my_data <-
read_csv("file.csv")
```

Work with Excel files

```
my_data <-
read_excel("file.xlsx")
```

Work with Stata, SPSS, and SAS data

```
my_data <-
read_stata("file.dta")
```

Data from R-Packages

Some data sets can be downloaded as packages in R. For example, the gapminder data set.

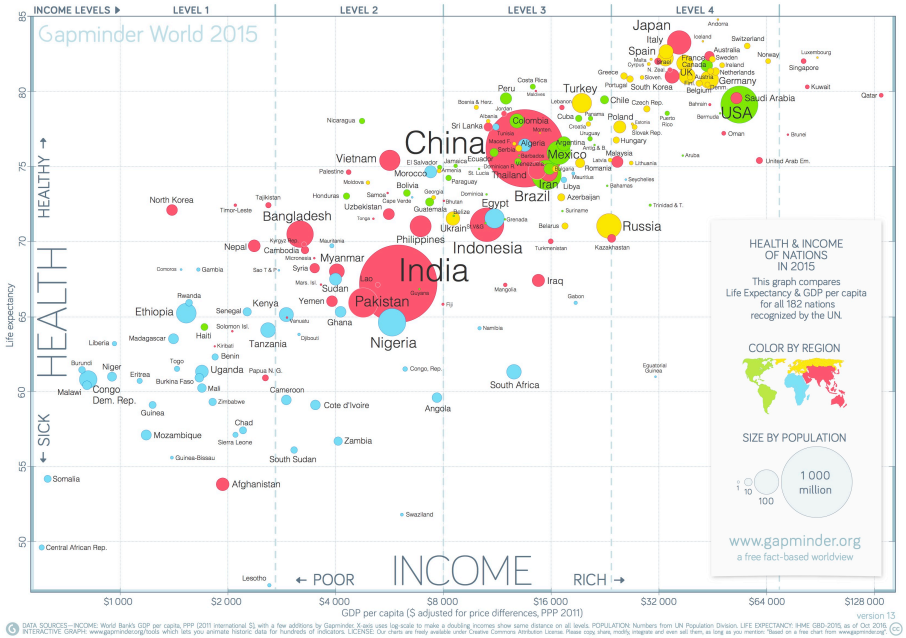
Install the package

```
install.packages(gapminder)
```

Then load the data

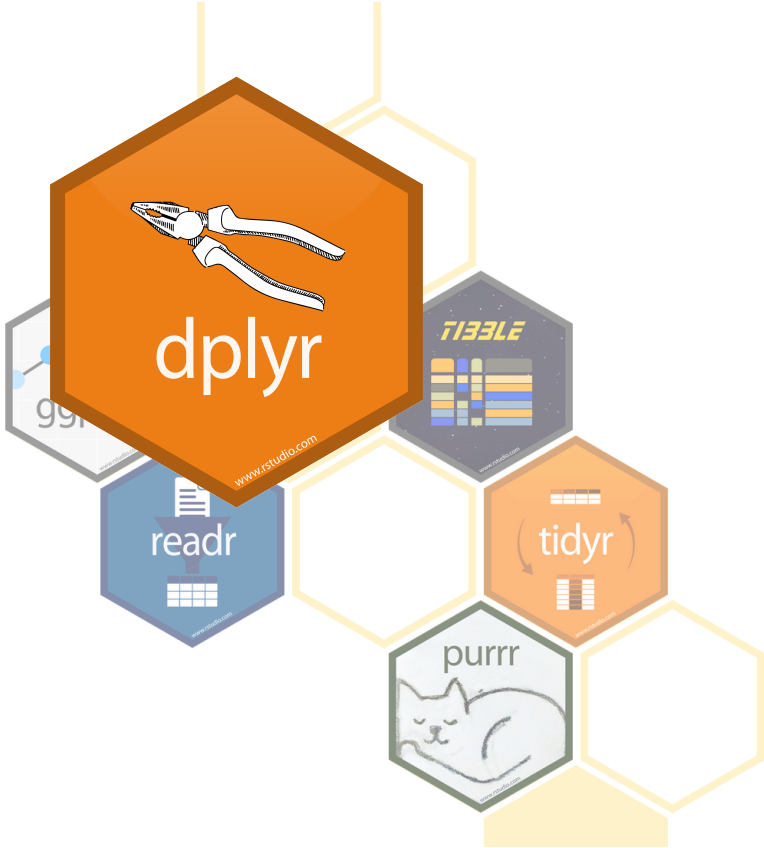
```
library(gapminder)

# The data() function in R is used to list, load,
# and access built-in or package-provided datasets
data(gapminder)
```



The `dplyr` package

tidyverse

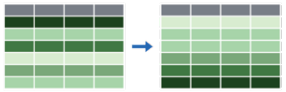


`dplyr`: verbs for manipulating data

Extract rows with `filter()`

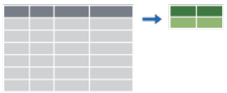
Extract columns with `select()`

Arrange/sort rows with `arrange()`



Make new columns with `mutate()`

Make group summaries with
`group_by() |> summarize()`



`filter()`

Extract rows that meet some sort of test

The general idea:

```
filter(
  some_data,
  ... # one or more tests
  )
```

Let's try this on the gapminder data set that you've installed earlier.

```
filter(.data = gapminder, country == "Denmark")
```

country	continent	year
Denmark	Europe	1952
Denmark	Europe	1957
Denmark	Europe	1962
Denmark	Europe	1967
Denmark	Europe	1972
...	...	...

Logical tests

Test	Meaning	Test	Meaning
`x < y`	Less than	`x %in% y`	In (group membership)
`x > y`	Greater than	`is.na(x)`	Is missing
`==`	Equal to	`!is.na(x)`	Is not missing
`x <= y`	Less than or equal to		
`x >= y`	Greater than or equal to		
`x != y`	Not equal to		

Your turn #1: Filtering

04:00

Use `filter()` and logical tests to show…

1. The data for Canada
2. All data for countries in Oceania
3. Rows where the life expectancy is greater than 82

Your turn #1: Filtering

Use `filter()` and logical tests to show…

1. The data for Canada

```
filter(gapminder, country == "Canada")
```

2. All data for countries in Oceania

```
filter(gapminder, continent == "Oceania")
```

3. Rows where the life expectancy is greater than 82

```
filter(gapminder, lifeExp > 82)
```

Common Mistakes

Using = instead of ==

Bad

```
filter(gapminder, country = "Canada")
```

Good

```
filter(gapminder, country == "Canada")
```

Forgetting quotes ("")

Bad

```
filter(gapminder, country == Canada)
```

Good

```
filter(gapminder, country == "Canada")
```

`filter()` with multiple conditions

Extract rows that meet *every* test

```
filter(gapminder, country == "Denmark", year > 2000)
```

```
# A tibble: 2 × 6
  country continent  year lifeExp     pop gdpPercap
  <fct>   <fct>     <int>   <dbl>   <int>     <dbl>
1 Denmark Europe     2002    77.2 5374693    32167.
2 Denmark Europe     2007    78.3 5468120    35278.
```

Boolean operators

Operator	Meaning
`a & b`	and
`a | b`	or
`!a`	not

Boolean operators

The default is “and”

These do the same thing:

```
filter(gapminder,
       country == "Denmark",
       year > 2000)
```

```
filter(gapminder,
       country == "Denmark" &
         year > 2000)
```

Your turn #2: Filtering

04:00

Use `filter()` and Boolean logical tests to show...

1. Canada before 1970
2. Countries where life expectancy in 2007 is below 50
3. Countries where life expectancy in 2007 is below 50 and are not in Africa

Your turn #2: Filtering

Use `filter()` and Boolean logical tests to show…

1. Canada before 1970

```
filter(gapminder, country == "Canada", year < 1970)
```

2. Countries where life expectancy in 2007 is below 50

```
filter(gapminder, year == 2007, lifeExp < 50)
```

3. Countries where life expectancy in 2007 is below 50 and are not in Africa

```
filter(gapminder, year == 2007, lifeExp < 50,
       continent != "Africa")
```

Common Mistakes

Collapsing multiple tests into one

Bad

```
filter(gapminder,
       1960 < year < 1980)
```

Good

```
filter(gapminder,
       year > 1960,
       year < 1980)
```

Using multiple tests instead of `%in%`

Bad

```
filter(gapminder,
       country == "Mexico",
       country == "Canada",
       country == "United States")
```

Good

```
filter(gapminder,
       country %in% c("Mexico", "Canada",
                      "United States"))
```

Common Syntax

Every `dplyr` verb function follows the same pattern

```
verb(data, ...)
```

`verb` = dplyr function/verb

`data` = data frame to transfom

`...` = what you the verb to do exatly

mutate()

Create new columns

The general idea:

```
mutate(
  some_data,
  ... # new columns to make
  )
```

Let's try this on the gapminder data

```
mutate(gapminder, gdp = gdpPercap * pop)
```

country	year	...	gdp
Afghanistan	1952	...	6567086330
Afghanistan	1957	...	7585448670
Afghanistan	1962	...	8758855797
Afghanistan	1967	...	9648014150
Afghanistan	1972	...	9678553274
Afghanistan	1977	...	11697659231

mutate()

Create new columns

The general idea:

```
mutate(
  some_data,
  ... # new columns to make
  )
```

We can also create multiple new columns at once

```
mutate(gapminder, gdp = gdpPercap * pop,
                  pop_mil = round(pop / 1000000))
```

country	year	...	gdp
Afghanistan	1952	...	6567086330
Afghanistan	1957	...	7585448670
Afghanistan	1962	...	8758855797
Afghanistan	1967	...	9648014150
Afghanistan	1972	...	9678553274
Afghanistan	1977	...	11697659231

`ifelse()`

Do conditional tests within `mutate()`

```
ifelse(test,
       value_if_true,
       value_if_false)
```

`test` = a logical test

`value_if_true` = what happens if test is true

`value_if_false` = what happens if test is false

ifelse()

The new variable can take any sort of class

```
# a new logical variable
mutate(gapminder,
       after_1960 = ifelse(year > 1960, TRUE, FALSE)
       )
```

```
# a new character variable
mutate(gapminder,
       after_1960 = ifelse(year > 1960,
                           "After 1960",
                           "Before 1960")
       )
```

```
# a new numeric variable
mutate(gapminder,
       after_1960 = ifelse(year > 1960, 0, 1)
       )
```

Your turn #3: Mutating

05:00

Use `mutate()` to...

1. Add an `africa` column that is TRUE if the country is on the African continent
2. Add a column for logged GDP per capita (hint: use `log()`)
3. Add an `africa_asia` column that says "Africa or Asia" if the country is in Africa or Asia, and "Not Africa or Asia" if it's not

Your turn #3: Mutating

Use `mutate()` to...

1. Add an `africa` column that is TRUE if the country is on the African continent

```
mutate(gapminder, africa = ifelse(continent == "Africa",
                                  TRUE, FALSE))
```

2. Add a column for logged GDP per capita (hint: use `log()`)

```
mutate(gapminder, log_gdpPercap = log(gdpPercap))
```

3. Add an `africa_asia` column that says "Africa or Asia" if the country is in Africa or Asia, and "Not Africa or Asia" if it's not

```
mutate(gapminder,
       africa_asia =
         ifelse(continent %in% c("Africa", "Asia"),
                "Africa or Asia",
                "Not Africa or Asia"))
```

What if you have multiple verbs?

Solution 1: Intermediate variables

```
gapminder_2002 <- filter(gapminder, year == 2002)

gapminder_2002_log <- mutate(gapminder_2002,
                             log_gdpPercap = log(gdpPercap))
```

Solution 2: Nested functions

```
filter(mutate(gapminder_2002,
              log_gdpPercap = log(gdpPercap)),
       year == 2002)
```

Solution 3: Pipes!

- The `|>` operator (pipe) takes an object on the left and passes it as the first argument of the function on the right

```
gapminder |>
  filter(year == 2002) |>
  mutate(log_gdpPercap = log(gdpPercap))
```

|>

Why using pipes?

```
leave_house(get_dressed(get_out_of_bed(wake_up(me, time = "8:00"), side = "correct"),
    pants = TRUE, shirt = TRUE), car = TRUE, bike = FALSE)
```

... 🤯 not easy to read

```
me |>
  wake_up(time = "8:00") |>
  get_out_of_bed(side = "correct") |>
  get_dressed(pants = TRUE, shirt = TRUE) |>
  leave_house(car = TRUE, bike = FALSE)
```

... 🎉 easy to read

`|>` vs `%>%`

- There are actually multiple pipes!
- `%>%` was invented first, but requires a package to use
- `|>` is part of base R
- They're interchangeable 99% of the time (Just be consistent)

You do not have to type the pipe by hand every time

You can use the shortcut `cmd + shift + m` in R Studio.

summarize()

Compute a table of summaries

1. Take a data frame

country	continent	year	lifeExp
Afghanistan	Asia	1952	28.801
Afghanistan	Asia	1957	30.332
Afghanistan	Asia	1962	31.997
Afghanistan	Asia	1967	34.02
…	…	…	…

2. Make a summary

```
gapminder |>
    summarize(mean_life = mean(lifeExp))
```

```
# A tibble: 1 × 1
  mean_life
      <dbl>
1      59.5
```

Or several summaries

```
gapminder |>
  summarize(mean_life = mean(lifeExp),
            min_life = min(lifeExp))
```

```
# A tibble: 1 × 2
  mean_life min_life
      <dbl>    <dbl>
1      59.5     23.6
```

Your turn #4: Summarizing

05:00

Use `summarize()` to calculate...

1. The first (minimum) year in the dataset
2. The last (maximum) year in the dataset
3. The number of rows in the dataset (use the cheatsheet)
4. The number of distinct countries in the dataset (use the cheatsheet)

Your turn #4: Summarizing

One Solution for all:

```r
gapminder |>
  summarize(first = min(year),
            last = max(year),
            num_rows = n(),
            num_unique = n_distinct(country))
```

```
# A tibble: 1 × 4
  first  last num_rows num_unique
  <int> <int>    <int>      <int>
1  1952  2007     1704        142
```

Your turn #5: Summarizing

05:00

Use `filter()` and `summarize()` to calculate...

1. the number of unique countries and
2. the median life expectancy

on the African continent in 2007.

Your turn #5: Summarizing

Use `filter()` and `summarize()` to calculate...

1. the number of unique countries and
2. the median life expectancy

on the African continent in 2007.

```
gapminder |>
  filter(continent == "Africa", year == 2007) |>
  summarise(n_countries = n_distinct(country),
            med_le = median(lifeExp))
```

```
# A tibble: 1 × 2
  n_countries med_le
        <int>  <dbl>
1          52   52.9
```

`group_by()`

Put rows into groups based on values in a column

```
gapminder |> group_by(continent)
```

- Nothing happens by itself!
- Powerful when combined with `summarize()`

group_by()

country	continent	year	lifeExp
Afghanistan	Asia	1952	28.801
Afghanistan	Asia	1957	30.332
Afghanistan	Asia	1962	31.997
Afghanistan	Asia	1967	34.02
…	…	…	…

A simple summary

```
gapminder |>
    summarize(n_countries = n_distinct(country))
```

```
# A tibble: 1 × 1
  n_countries
        <int>
1         142
```

A grouped summary

```
gapminder |>
  group_by(continent) |>
  summarize(n_countries = n_distinct(country))
```

```
# A tibble: 5 × 2
  continent n_countries
  <fct>           <int>
1 Africa             52
2 Americas           25
3 Asia               33
4 Europe             30
5 Oceania             2
```

Your turn #6: Grouping and summarizing

1. Find the minimum, maximum, and median life expectancy for each continent

2. Find the minimum, maximum, and median life expectancy for each continent in 2007 only

05:00

Your turn #6: Grouping and summarizing

1. Find the minimum, maximum, and median life expectancy for each continent

```
gapminder |>
  group_by(continent) |>
  summarize(min_le = min(lifeExp),
            max_le = max(lifeExp),
            med_le = median(lifeExp))
```

2. Find the minimum, maximum, and median life expectancy for each continent in 2007 only

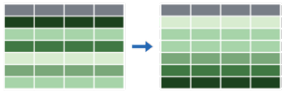
```
gapminder |>
  filter(year == 2007) |>
  group_by(continent) |>
  summarize(min_le = min(lifeExp),
            max_le = max(lifeExp),
            med_le = median(lifeExp))
```

`dplyr`: verbs for manipulating data

Extract rows with `filter()`

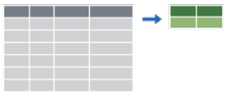
Extract columns with `select()`

Arrange/sort rows with `arrange()`



Make new columns with `mutate()`

Make group summaries with
`group_by() |> summarize()`



Merge and re-shape